Laboratory Exercise 11
General-Purpose Microprocessor (EC-1 and EC-2)

Introduction

In Lab 10 you built a *dedicated* microprocessor: its datapath and control unit were designed for one algorithm only. A *general-purpose* microprocessor runs *any* program, because the algorithm is stored in memory as a sequence of instructions, and the control unit simply fetches, decodes and executes them (Lecture 10). In this laboratory you will work with the two processors of the lecture. The **EC-1** is an 8-bit processor with a read-only program memory and only five instructions. The **EC-2** adds a read/write memory shared by code and data (the Von Neumann architecture), an adder/subtractor and more instructions. You will run existing programs, write your own programs in machine code, simulate the processors, and run them on the DE0-CV board.

Objectives

1. To understand the fetch–decode–execute instruction cycle by observing it in a simulation.
2. To encode assembly programs into machine code and to store them in memory using the Intel HEX format.
3. To run and debug programs on the EC-1 in simulation and on the DE0-CV board.
4. To write a program for the EC-2, a stored-program computer in which code and data share one memory.
5. To understand why a program can modify other programs (or itself) in a Von Neumann machine.

Equipment Required

- Intel DE0-CV board and a PC with Quartus Prime and the simulator selected by your instructor
- Starter files supplied by your instructor (course repository, folder `vhdl`): EC-1 files `ec1.vhd`, `ec1_components.vhd`, `ec1_datapath.vhd`, `ec1_control_unit.vhd`, `ec1_tb.vhd`, `d_ff.vhd`, `rom_data_1.hex`; EC-2 files `ec2.vhd`, `EC2_Components.vhd`, `EC2_Datapath.vhd`, `EC2_Control_Unit.vhd`, `ec2_tb.vhd`
- `hex7seg.vhd` from Lab 4

Pre-lab (complete before the session)

The session is two hours long, so the encoding work is done in advance. Complete the following *before* the session, and also the first column of Table 4 (Part III).

Instruction sets

Both processors use an 8-bit instruction: the upper 3 bits are the **opcode** and the lower bits are an operand (an address) or are not used (x = don't care, write 0).

EC-1 instruction	Encoding	EC-2 instruction	Encoding
IN A	011 xxxxx	LOAD A, addr	000 aaaaa
OUT A	100 xxxxx	STORE A, addr	001 aaaaa
DEC A	101 xxxxx	ADD A, addr	010 aaaaa
JNZ addr	110 xaaaa	SUB A, addr	011 aaaaa
HALT	111 xxxxx	IN A	100 xxxxx
		JZ addr	101 aaaaa
		JPOS addr	110 aaaaa
		HALT	111 xxxxx

Table 1: Instruction encodings. EC-1: 4-bit addresses, 16-byte ROM. EC-2: 5-bit addresses, 32-byte RAM. JZ jumps if $A = 0$; JPOS jumps if $A \geq 0$.

Machine code and the Intel HEX file

A program is loaded into the memory from a text file in Intel HEX format. Each line has the form `:LLAAAATT[DD...]CC`: `LL` = number of data bytes, `AAAA` = start address, `TT` = record type (`00` data, `01` end of file), `DD` = data bytes and `CC` = checksum. The checksum is the two's complement of the (8-bit) sum of all the other bytes of the line, so that the sum of *all* bytes of the line, including the checksum, is 00_{16}.

1. Encode the following EC-1 program (countdown from the input value to 1) in hexadecimal:

Address	Assembly	Hex
00	IN A	
01	OUT A	
02	DEC A	
03	JNZ 01	
04	HALT	

2. Compute the checksum of the 5-byte data line `:05000000` *your five bytes* `CC`. Checksum = _____. Show your calculation.

3. The file `rom_data_1.hex` holds the same program in a 16-byte line, `:100000006080A0C1E0...`. Verify that its checksum `CF` is correct. (Hint: the zeros do not change the sum.)

Part I: The EC-1 in Simulation (about 30 min)

1. Create a new Quartus project `lab11` and add all EC-1 files. The EC-1 program memory is a ROM called `rom32x8`, which is a megafunction (IP core). Create it with the IP Catalog: **ROM: 1-PORT**, data width 8 bits, 32 words, single clock, **no** registered output, and initial contents `rom_data_1.hex`. The ports must be named `address`, `clock` and `q`. Set `ec1` as the top-level entity and compile.

2. Simulate the testbench `ec1_tb`, which resets the processor and applies the input value 10 ($0A_{16}$). Show these signals in the waveform: `clk`, `reset`, `dbg_state`, `dbg_PC`, `dbg_IR`, `input_bus`, `output_bus` and `halt`. Run the simulation until `halt` becomes 1.

3. Observe the instruction cycle of each instruction in your waveform. List the control unit states that are visited and the number of clock cycles used by each instruction (Table 2). In the VHDL control unit, state codes are 000 = Fetch, 001 = Decode, 011 = Input, 100 = Output, 101 = Dec, 110 = JNZ, 111 = Halt. One state code is not in the lecture's FSM. Which one is it, and in which instruction is it used? _____

4. **Predict, then measure.** Using Table 2, write a formula for the number of clock cycles from the release of reset until `halt` = 1 as a function of the input N: $T(N) =$ _____. Verify your formula in the simulation for $N = 10$ and for another value (change `input_bus` in the testbench).

5. Explain why `dbg_PC` is already "next address" while an instruction executes (see the state in which `PCload` is active).

Instruction	**States visited (in order)**	**Clock cycles**
IN A	Fetch, Decode, Input	3
OUT A		
DEC A		
JNZ (jump taken)		
JNZ (jump not taken)		
HALT (to reach the Halt state)		

Table 2: Instruction cycle of the EC-1.

Part II: The EC-1 on the DE0-CV Board (about 35 min)

To run the EC-1 on the board you need a top-level entity (Figure 1). The 50 MHz clock is too fast to watch, so the processor clock is selected by SW9: either a slow clock of about 1.5 Hz obtained from a counter ($50\,\text{MHz}/2^{25}$), or a *single-step* clock given by the push button KEY0. KEY1 is the (active-high) reset of the processor after inversion. Note that the output of the EC-1 is a **tri-state bus**: it is driven only during the Output state and is high-impedance otherwise (the same problem as in Lab 10). Therefore the value is stored in a register at the end of the Output state, and the register is displayed on HEX1–HEX0.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY ec1_top IS
    PORT ( CLOCK_50  : IN  STD_LOGIC;
           KEY       : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
           SW        : IN  STD_LOGIC_VECTOR(9 DOWNTO 0);
           LEDR      : OUT STD_LOGIC_VECTOR(9 DOWNTO 0);
           HEX0, HEX1 : OUT STD_LOGIC_VECTOR(6 DOWNTO 0) );
END ec1_top;

ARCHITECTURE structural OF ec1_top IS
    SIGNAL cnt               : UNSIGNED(24 DOWNTO 0) := (OTHERS => '0');
    SIGNAL clk_cpu, reset    : STD_LOGIC;
    SIGNAL halt              : STD_LOGIC;
    SIGNAL bus_out, out_reg  : STD_LOGIC_VECTOR(7 DOWNTO 0) := (OTHERS => '0');
    SIGNAL pc                : STD_LOGIC_VECTOR(3 DOWNTO 0);
    SIGNAL st                : STD_LOGIC_VECTOR(2 DOWNTO 0);
BEGIN
    PROCESS (CLOCK_50)
    BEGIN
        IF rising_edge(CLOCK_50) THEN cnt <= cnt + 1; END IF;
    END PROCESS;

    clk_cpu <= cnt(24) WHEN SW(9) = '1' ELSE NOT KEY(0);  -- run / single step
    reset   <= NOT KEY(1);

    cpu : ENTITY work.ec1
        PORT MAP ( clk => clk_cpu, reset => reset, input_bus => SW(7 DOWNTO 0),
                   output_bus => bus_out, halt => halt, dbg_opcode => OPEN,
                   dbg_A_neq_0 => OPEN, dbg_PC => pc, dbg_IR => OPEN, dbg_state => st );

    -- capture the output bus at the end of the Output state (state = "100")
    PROCESS (clk_cpu)
    BEGIN
        IF rising_edge(clk_cpu) THEN
            IF st = "100" THEN out_reg <= bus_out; END IF;
        END IF;
    END PROCESS;

    -- TODO: hex7seg instances for HEX1 (out_reg(7 DOWNTO 4)) and HEX0 (out_reg(3 DOWNTO 0))
    -- TODO: LEDR(3 DOWNTO 0) <= pc;  LEDR(6 DOWNTO 4) <= st;  LEDR(9) <= halt;
END structural;
```

Figure 1: Top-level entity for the EC-1 (complete the `TODO` lines).

1. Complete the top-level entity, assign pins (SW, KEY, LEDR, HEX, CLOCK_50) and compile. Record the number of logic elements used: _____.

2. Set SW7–SW0 = 00001010, set SW9 = 0, press KEY1 to reset and single-step the processor with KEY0. Write the LED pattern (PC and state) after each press for the first 12 presses in Table 3. Then set SW9 = 1 and watch the countdown on HEX1–HEX0.

3. Test the program with another input (for example 5 and 1). What is the display value when the processor has halted? Which LED indicates the halt?

4. The clock of the processor is ≈ 1.5 Hz. How long does it take to run the countdown from 10? Does it agree with your formula $T(N)$ of Part I?

Press	PC (LEDR3–0)	State (LEDR6–4)
1		
2		
3		
...		
12		

Table 3: Single-step trace of the EC-1.

Part III: The EC-2 and the GCD Program (about 35 min)

The EC-2 has a single 32-byte RAM that holds *both* the program and the data. The address of the RAM is selected by the program counter during the fetch and by the instruction register during LOAD, STORE, ADD and SUB (control signal `MemInst`). The following program computes the greatest common divisor of X and Y by repeated subtraction (Euclid's algorithm), where X is stored at address 30 and Y at address 31:

```
while (X != Y) {
    if (X > Y) X = X - Y;
    else       Y = Y - X;
}
```

Addr (dec)	Assembly	Comment	Binary	Hex
00	LOAD A, 30	$A \leftarrow X$		
01	SUB A, 31	$A \leftarrow X - Y$		
02	JZ 10	if $X = Y$, finished		
03	JPOS 08	if $X > Y$, update X		
04	LOAD A, 31	else $A \leftarrow Y$		
05	SUB A, 30	$A \leftarrow Y - X$		
06	STORE A, 31	$Y \leftarrow A$		
07	JPOS 00	loop		
08	STORE A, 30	$X \leftarrow A$		
09	JPOS 00	loop		
10	HALT	result in Mem[30]		

Table 4: GCD program for the EC-2 (addresses are in decimal).

1. Complete Table 4 with the binary and hexadecimal machine code (Table 1). Compare it with the initial contents of the array in the entity `ram32x8` of `EC2_Components.vhd`. Check that the program and the data ($X = 15$ at address 30 and $Y = 25$ at address 31) agree with your table.

2. Create a project for the EC-2 (top-level entity `ec2`). Compile it and simulate `ec2_tb`. Show `dbg_state`, `dbg_PC`, `dbg_IR`, `output_bus` and `halt`. Observe the sequence of addresses visited by the program counter. Which addresses are executed for $X = 15$, $Y = 25$? Write the sequence: _____

3. The EC-2 has no instruction to output memory data, and the output bus shows the accumulator A (which is zero when the program stops through `JZ`). Modify the program so that the result is visible: replace the HALT at address 10 with `LOAD A, 30`, and put the HALT at address 11. (The `JZ 10` still points to the correct place.) Write the two new hex bytes: _____. Check in the simulation that `output_bus` $= 5$ when `halt` $= 1$.

4. Test the program with the data pairs of Table 5 by changing the initial values of addresses 30 and 31 in `ram32x8` (in simulation).

5. Reuse the top-level of Figure 1 for the EC-2: the program counter is 5 bits and `dbg_state` is 4 bits, and `A_out_bus` (`output_bus`) is *not* tri-state, so the capture register is not needed. Display `output_bus` on HEX1–HEX0, the halt signal on LEDR9 and the lower bits of the PC on LEDR4–LEDR0. Run the GCD program on the board.

X	Y	**Expected GCD**	**HEX1–HEX0 at halt**
15	25	5	
48	18		
7	13		
12	12		

Table 5: GCD results.

Extension (take-home, or if time permits)

EC-1 Odd/Even Detector

The program halts at address 03 if the input is odd and at address 06 if it is even. It uses only DEC and JNZ: it "ping-pongs" between two decrements, so it checks one bit without any bit-test instruction.

Address	**Assembly**	**Hex**
00	IN A	
01	DEC A	
02	JNZ 04	
03	HALT ; halts here if odd	
04	DEC A	
05	JNZ 01	
06	HALT ; halts here if even	

1. Complete the table, write the HEX line, compute its checksum, and test on the board with the inputs of Table 6. Use the PC value on the LEDs when the processor halts to decide odd/even, and explain why the PC is one more than the address of the HALT instruction.
2. What happens with input 0? Predict first, then test. Explain the result in terms of the 8-bit register.

Input	**PC at halt**	**Odd or even?**	**Correct?**
7			
12			
1			
0			

Table 6: Results of the odd/even detector.

GCD, again: why must all the numbers be below 128 for the GCD program to work? (Hint: look at how JPOS decides.)

Write Your Own EC-2 Program

Write an EC-2 program that computes the **product** $P = X \times Y$ of two numbers by *repeated addition* (Lecture 10, Challenge 3): $P \leftarrow 0$; while $Y \neq 0$: $P \leftarrow P + X$, $Y \leftarrow Y - 1$. Use the following memory map:

Address	**Content**
0 – 15	program
28	P (product), initial value 0
29	constant 1
30	X
31	Y

1. Write the program in assembly language with comments. Hints: the EC-2 can only subtract a *memory* operand, so the constant 1 is stored at address 29; the instruction `JPOS` behaves like an unconditional jump when you know that $A \geq 0$.

2. Convert it to machine code, enter it in the RAM initialization of `ram32x8` together with the data, and make the program end with the product in the accumulator (as in Part III).

3. Simulate with $X = 6$, $Y = 7$ and then with $Y = 0$. Does the program give 42 and 0? How many clock cycles does it need for each case?

4. Test the program on the board. What is the largest product that it can compute correctly? Why?

Code Is Data

Because the program and the data share one memory, a program can modify its own instructions. The bootloader below loads bytes from the input port to consecutive memory addresses starting at address 10, although the EC-2 has no indirect addressing.

Addr	Assembly	Comment
00	IN A	read a byte from the input
01	STORE A, 10	store it at the target address (this instruction will be modified)
02	LOAD A, 01	load the instruction at address 01
03	ADD A, 06	add 1 to its address field
04	STORE A, 01	write it back
05	JPOS 00	next byte
06	01	constant 1 (data)

1. What is the machine code of `STORE A, 10` at address 01? What does it become after one pass of the loop? After two passes? Which instruction does the second one correspond to?

   _____

2. Why does `JPOS 00` always jump back here? (Hint: what is the value of A at this point, and what is the opcode of STORE?)

3. **Optional challenge.** How would you add an instruction `INC A` ($A \leftarrow A + 1$) to the EC-1, using the unused opcode 001? List the changes in the datapath and in the control unit. How many state flip-flops does the control unit need then, and why?

Check-off

Show your instructor or teaching assistant:

1. Your hand-encoded programs and the HEX checksum (Pre-lab).
2. The EC-1 countdown in simulation (Table 2) and on the board (Table 3).
3. The EC-2 GCD program in simulation and on the board (Table 5).

Updated By: R. Sutthaweekul
Version: 1.1
Release Date: 2026-11-11